

UOD-DSM Implementation Report

Unified Object Detector with Dynamic Switch Mechanism

Purpose: Explain the implemented concept, code architecture, baseline configuration, modified parameters, execution flow, evaluation and limitations.

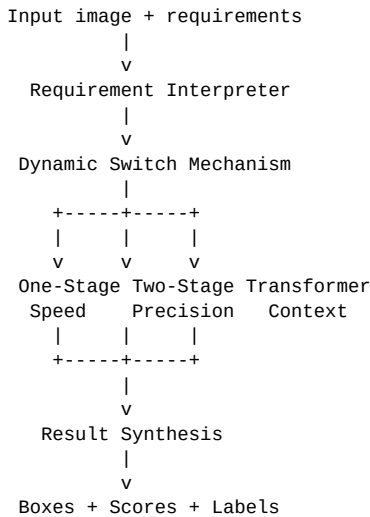
Item	Implementation
Framework	Python + PyTorch + Torchvision
Backbone	ResNet-18 feature extractor
Stage I	Requirement Interpreter
Stage II	Rule-based Dynamic Switch Mechanism
Stage III	One-stage, two-stage and transformer heads
Stage IV	Result Synthesis Module
Data	Synthetic multi-domain object-detection data
Experiments	Baseline vs modified parameters

Scope: This is a proof-of-concept implementation of the architecture and switching methodology described in the paper. It is not presented as an exact reproduction of an official author-released, fully trained detector.

1. Concept and Objective

The UOD-DSM idea is to avoid using one fixed object-detection paradigm for every situation. The system dynamically selects a detection paradigm according to operational requirements and scene characteristics.

Main mapping: one-stage detection is treated as speed-oriented, two-stage detection as precision-oriented, and transformer detection as context/complexity-oriented.



The implementation follows the four-stage structure described in the paper: Stage I interprets requirements, Stage II selects the paradigm, Stage III executes the selected detector, and Stage IV produces standardized output.

2. Code Structure

Code component	Role
DomainConfig	Stores domain requirements such as latency, precision and complexity
MultiDomainDetectionDataset	Generates synthetic images, boxes, labels and metadata
SharedBackbone	Extracts shared visual features using ResNet-18
OneStageHead	Speed-oriented detector head
TwoStageHead	Proposal and refinement detector head
TransformerHead	Query-based context detector head
RequirementInterpreter	Processes metadata and visual features
DynamicSwitchMechanism	Selects the detection paradigm
ResultSynthesisModule	Filters and standardizes final detections
UnifiedObjectDetector_DSM	Combines the complete four-stage pipeline

3. Domain Configuration and Synthetic Dataset

The current implementation uses a synthetic multi-domain dataset. Each generated sample contains an image, bounding boxes, class labels, a domain identifier and a three-value metadata vector.

Domain	Requirement represented	Expected routing
Autonomous / Real-Time	Speed / low latency	One-stage
Medical Imaging	High precision	Two-stage
Remote Sensing / Aerial	High scene complexity	Transformer

The routing metadata is represented as: **[latency budget, required precision, scene complexity]**. This metadata is what drives the rule-based DSM decision.

```
metadata = [  
    latency_budget,  
    required_precision,  
    scene_complexity  
]
```

The use of synthetic data is important when interpreting the results: the current code demonstrates the architecture and routing behavior, rather than providing a real-world object-detection benchmark.

4. Shared Feature Extraction

The SharedBackbone uses ResNet-18 from Torchvision. The classification portion is not used; the early and intermediate residual layers produce a feature map shared by all detector branches.

```
resnet = torchvision.models.resnet18(weights=None)  
self.stem = nn.Sequential(  
    resnet.conv1, resnet.bn1, resnet.relu, resnet.maxpool  
)  
self.layer1 = resnet.layer1  
self.layer2 = resnet.layer2  
self.layer3 = resnet.layer3
```

5. Stage III — Detection Heads

The implementation contains three detector branches. They share the same backbone features but differ in their detection mechanism.

5.1 OneStageHead

The one-stage branch applies convolutional layers and directly predicts class probabilities and bounding boxes. It represents the speed-oriented paradigm.

5.2 TwoStageHead

The two-stage branch first creates region proposals, selects the highest-scoring proposals, performs RoI pooling, and then predicts classes and refined boxes. It represents the precision-oriented paradigm.

5.3 TransformerHead

The transformer branch projects the feature map into transformer dimensions, uses positional embeddings and learnable object queries, and applies transformer decoder layers before predicting classes and boxes. It represents the context-oriented paradigm.

Branch	Main mechanism	Conceptual role
One-stage	Direct dense prediction	Speed
Two-stage	Proposal + RoI refinement	Precision
Transformer	Object queries + attention	Context

6. Stage I — Requirement Interpreter

The RequirementInterpreter processes the three operational metadata values using a small MLP. It can also process the shared visual feature map through global pooling and fully connected layers. These representations are fused to produce three routing logits.

```
Metadata -----+
                  |
Visual features -----+--> Fusion --> 3 routing logits
```

In the current experiment the DSM is configured in **rule_based** mode, so the final paradigm decision is made from the metadata thresholds rather than the neural routing logits.

7. Stage II — Dynamic Switch Mechanism

The DynamicSwitchMechanism is the central adaptive component. Its rule-based routing checks the requirements in sequence.

```
if latency < latency_threshold:
    one_stage
elif precision > precision_threshold:
    two_stage
elif complexity > complexity_threshold:
    transformer
...
```

The configurable thresholds allow the experiment to investigate how changing the routing criteria changes the selected detector paradigm.

8. Stage III — Activating the Selected Detector

The three detector heads are stored in a dictionary. The selected paradigm determines which head is executed. The evaluation uses batch size one, so each evaluated sample is routed independently.

9. Stage IV — Result Synthesis

ResultSynthesisModule provides a common output interface. It removes detections below the confidence threshold, sorts the remaining scores and stores a fixed maximum number of detections.

Output	Meaning
boxes	Predicted normalized bounding boxes
scores	Confidence scores
labels	Predicted class indices

10. UnifiedObjectDetector_DSM

This class combines the entire architecture. The forward pass performs the following sequence:

```
features = backbone(images)
routing_logits = requirement_interpreter(metadata, features)
decisions, paradigm_names = dsm(routing_logits, metadata)
raw_detections = selected_head(features)
final_output = result_synthesis(raw_detections)
```

The model also records L_I, L_II, L_III and L_IV. L_total is calculated as L_I + L_II + L_III + L_IV. Backbone timing is reported separately.

11. Baseline and Modified Parameters

The project keeps one implementation and runs it twice. The architecture is unchanged; only selected parameters are modified.

Parameter	Baseline	Modified	Effect
Latency threshold	0.30	0.40	Changes speed-priority routing
Precision threshold	0.80	0.75	Changes precision-priority routing
Complexity threshold	0.70	0.60	Changes transformer routing
Confidence threshold	0.10	0.30	Stricter output filtering
Two-stage proposals	32	64	More proposal candidates
Transformer queries	50	100	More object queries

Baseline configuration:

```
BASELINE_CONFIG = {  
    'latency_threshold': 0.30,  
    'precision_threshold': 0.80,  
    'complexity_threshold': 0.70,  
    'confidence_threshold': 0.10,  
    'num_proposals': 32,  
    'num_queries': 50,  
}
```

Modified configuration:

```
MODIFIED_CONFIG = {  
    'latency_threshold': 0.40,  
    'precision_threshold': 0.75,  
    'complexity_threshold': 0.60,  
    'confidence_threshold': 0.30,  
    'num_proposals': 64,  
    'num_queries': 100,  
}
```

This setup satisfies the requirement to implement the same methodology and then change selected parameters without changing the research architecture.

12. Evaluation

For each domain, the program evaluates multiple samples and records the following values:

Metric	Meaning
Average latency	Runtime cost of the adaptive pipeline
L_I	Requirement interpretation time
L_II	DSM routing time
L_III	Selected detector execution time
L_IV	Result synthesis time
Paradigm distribution	How often each paradigm is selected
Average detections	Number of retained detections per image
Average confidence	Mean confidence of retained detections

The program also benchmarks each detector as a static paradigm and compares it with the adaptive UOD-DSM result. The current implementation uses average confidence as a proxy statistic. It does not calculate standard object-detection metrics such as IoU, precision, recall or mAP.

13. End-to-End Execution Flow

1. Load libraries and select CPU/GPU.
2. Define domain requirements.
3. Generate synthetic domain data.
4. Extract shared features with ResNet-18.
5. Interpret metadata and visual features.
6. Apply DSM routing.
7. Execute the selected detection head.
8. Synthesize and filter the output.
9. Record stage latencies and detection statistics.
10. Run the baseline configuration.
11. Run the modified configuration.
12. Compare both configurations.
13. Benchmark static paradigms against adaptive execution.

14. Implementation Scope and Limitations

Area	Current implementation
Dataset	Synthetic multi-domain data
Training	Proof-of-concept inference pipeline; no full end-to-end training experiment
Detector heads	Custom lightweight implementations, not official YOLO/Faster R-CNN/DETR code
Accuracy	Average confidence is a proxy; no mAP/IoU evaluation
Routing	Main experiment uses rule-based DSM
Batch routing	A dominant paradigm is selected for a batch; evaluation uses batch size one
Latency	Depends on the execution hardware and runtime environment

These limitations define the scope of the proof-of-concept and should be stated rather than claiming that the code reproduces the paper's exact trained results.

15. Conclusion

The implementation converts the UOD-DSM research concept into an executable four-stage PyTorch pipeline. Its key contribution at the implementation level is the dynamic selection of detector paradigms based on operational requirements.

The baseline and modified experiments use the same architecture, making the parameter changes easy to isolate and explain. The resulting measurements can be used to discuss how routing thresholds and detector parameters affect system behavior.

Recommended project description: "Implementation of the UOD-DSM architecture and dynamic-switching methodology proposed in the research paper, followed by parameter modification and comparative evaluation."

Reference: O. Hmidani and E. M. Ismaili Alaoui, "Unified Object Detector With Dynamic Switch Mechanism Based on Deep Learning Advances: A Paradigm-Aware Architecture for Adaptive Detection," IEEE Access, 2026, DOI: 10.1109/ACCESS.2026.3680430.

Implementation source: UOD_DSM_Implementation_Baseline_Modified_NoComments_Fixed.ipynb